Machine Learning Operations (MLOps)

Assignment 01 — AIMLCZG523

End-to-End ML Model Development, CI/CD and Production Deployment

Problem: Predict the risk of heart disease from patient health data and deploy the solution as a cloud-ready, monitored API.

Dataset: Heart Disease UCI (Cleveland processed, 303 records)

Name: Goru Sai Sree Chaitanya

BITS ID: 2024AC05734

Total Marks: 50

1. Project Overview

This project implements a complete MLOps workflow for a binary classification problem: predicting the presence of heart disease. The solution covers the full lifecycle — data acquisition, exploratory data analysis, feature engineering, model development with hyperparameter tuning, experiment tracking with MLflow, reproducible packaging, automated testing and CI/CD, containerisation with Docker, deployment on Kubernetes with a Helm chart, and monitoring with Prometheus and Grafana.

1.1 Architecture

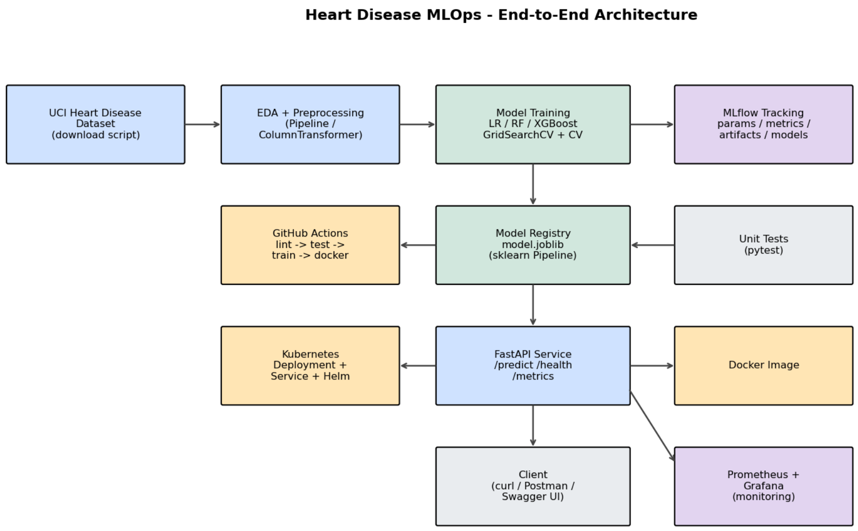

Figure 1: End-to-end MLOps architecture.

Tech stack: Python 3.12, Pandas/NumPy, Scikit-learn, XGBoost, Matplotlib/Seaborn, MLflow, FastAPI, Pytest, Docker, Kubernetes/Helm, Prometheus, Grafana, GitHub Actions.

1.2 Setup & Installation

The project installs cleanly into a fresh virtual environment from pinned dependencies. From the repository root:

```
python -m venv .venv
.venv\Scripts\activate  # Windows (source .venv/bin/activate on Linux/macOS)
pip install -r requirements.txt -r requirements-dev.txt
pip install -e .
python -m heart_mlops.data_download  # download UCI dataset -> data/raw/
python -m heart_mlops.eda  # EDA figures -> reports/figures/
python -m heart_mlops.train  # train + tune + MLflow -> models/model.joblib
mlflow ui --port 5000  # inspect experiments at http://localhost:5000
pytest  # run the 14 unit + integration tests
uvicorn api.main:app --port 8000  # serve the API locally
docker build -t heart-disease-api:latest .
docker run --rm -p 8000:8000 heart-disease-api:latest
kubectl apply -f k8s/  # or: helm install heart-api ./helm/heart-api
docker compose up -d  # Prometheus + Grafana monitoring stack
```

A pinned `requirements.txt` / `pyproject.toml` and a fixed random seed (42) make the setup fully reproducible on any clean machine.

2. Data Acquisition & Exploratory Data Analysis

The Cleveland processed dataset is downloaded from the UCI Machine Learning Repository via `data_download.py`. It contains 303 records with 13 clinical features and a target (raw 0–4, binarised to 0 = no disease, 1 = disease). Missing values in `ca` (4) and `thal` (2) are imputed. The classes are reasonably balanced (164 vs 139).

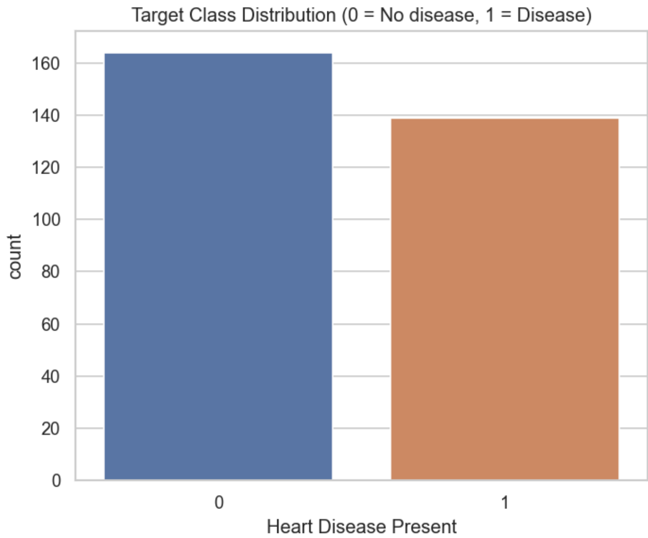

Figure 2: Target class distribution.

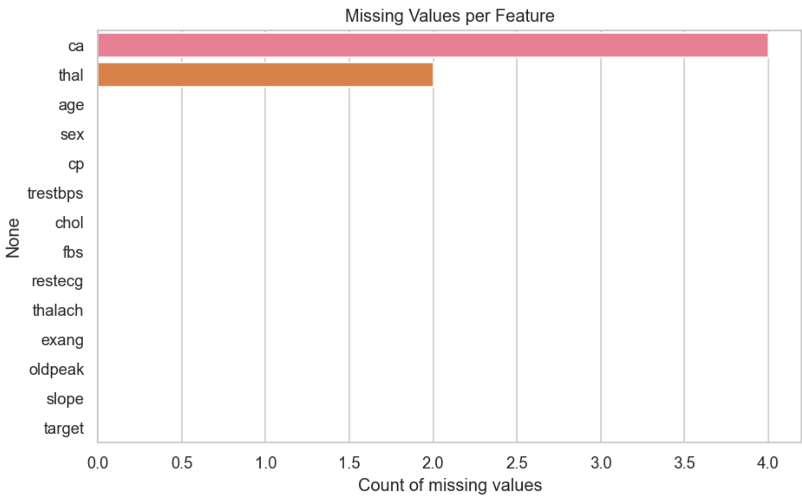

Figure 3: Missing-value analysis.

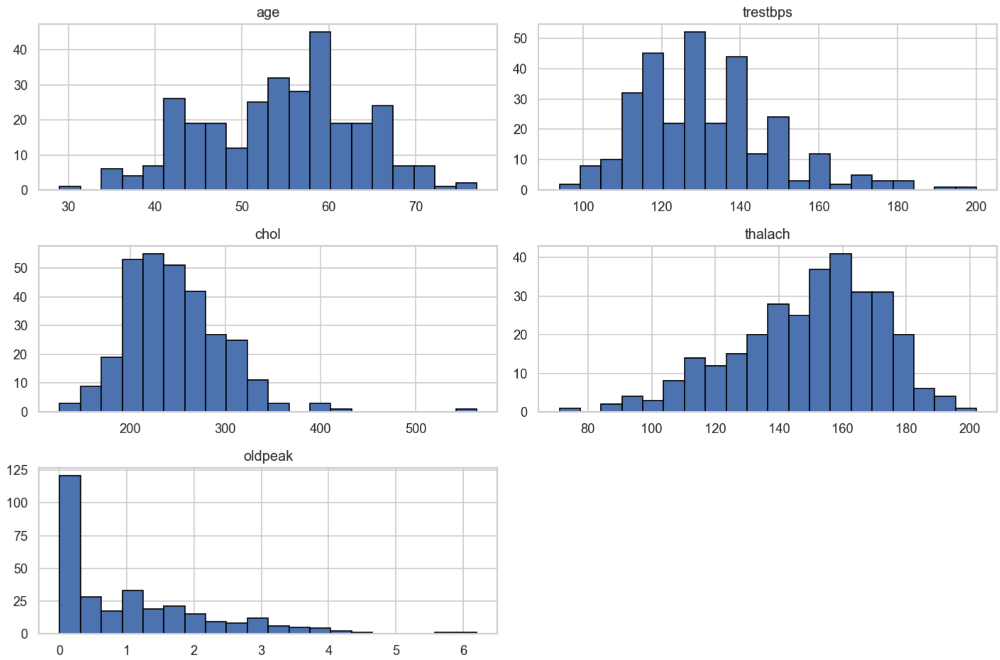

Figure 4: Distributions of numeric features.

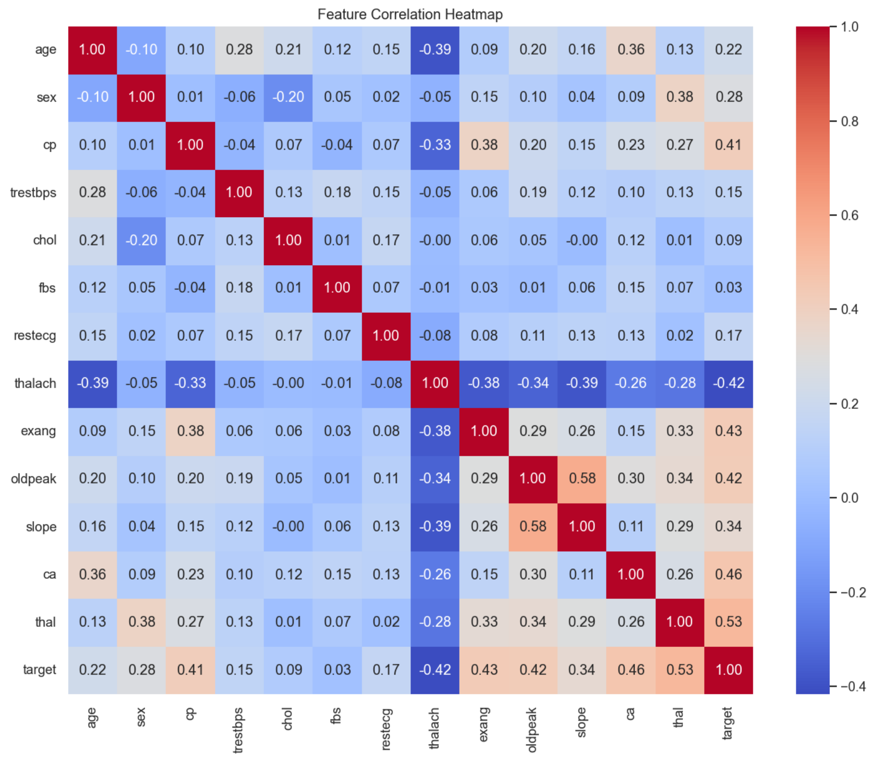

Figure 5: Correlation heatmap. cp, thalach, oldpeak, ca and thal show the strongest association with the target.

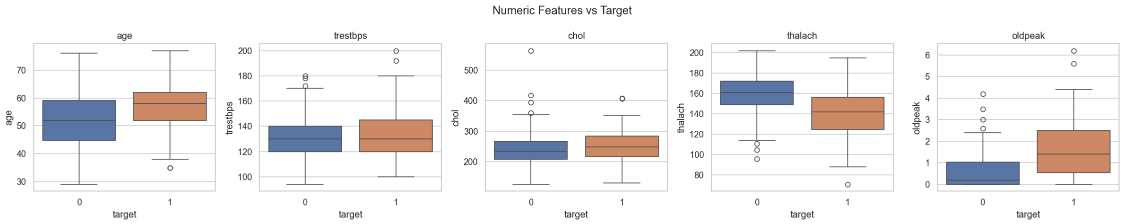

Figure 6: Numeric features vs target — patients with disease tend to show lower max heart rate (thalach) and higher oldpeak.

3. Feature Engineering & Model Development

Preprocessing is encapsulated in a scikit-learn `ColumnTransformer`: numeric features are median-imputed and standard-scaled; categorical features are most-frequent-imputed and one-hot encoded. This transformer is bundled with the classifier inside a single `Pipeline`, guaranteeing identical transformations at train and inference time.

Three classifiers were trained — Logistic Regression, Random Forest and XGBoost — each tuned with `GridSearchCV` over a hyperparameter grid using 5-fold stratified cross-validation, scored by ROC-AUC. Models were evaluated on a held-out 20% test set.

3.1 Model Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
Logistic Regression *	0.885	0.839	0.929	0.881	0.966	0.902+/-0.014
Random Forest	0.885	0.839	0.929	0.881	0.955	0.898+/-0.023
Xgboost	0.902	0.867	0.929	0.897	0.931	0.871+/-0.021

Table 1: Model comparison (* = selected model). Selected: **Logistic Regression** for the highest ROC-AUC.

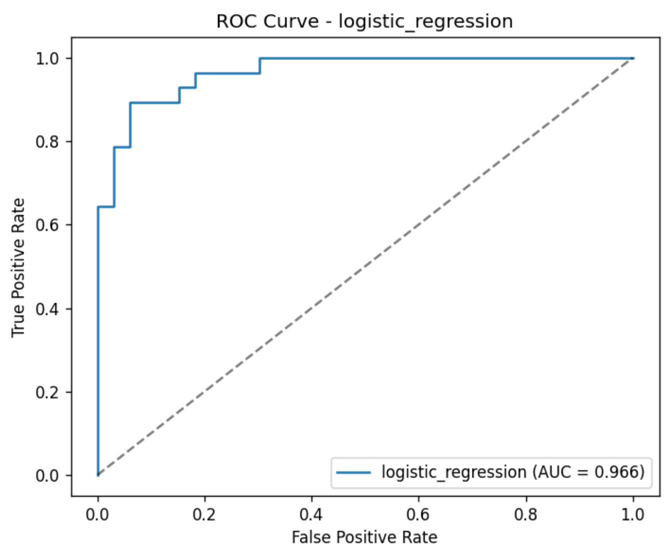

Figure 7: ROC curve of the selected model.

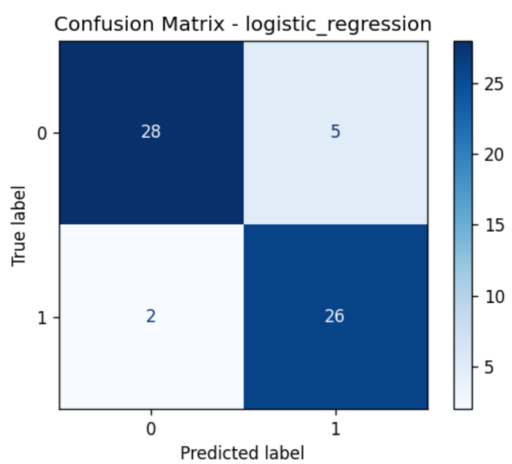

Figure 8: Confusion matrix of the selected model.

4. Experiment Tracking (MLflow)

Every training run is logged to MLflow under the experiment `heart-disease-classification`. For each model we log: hyperparameters (best GridSearchCV params, CV folds), metrics (accuracy, precision, recall, F1, ROC-AUC and cross-validated ROC-AUC), artifacts (ROC curve and confusion matrix PNGs) and the serialized sklearn model. The best model is additionally logged in a dedicated run. The MLflow UI (`mlflow ui`) provides side-by-side run comparison and full reproducibility.

Screenshots below (Figures 9–10) show the live MLflow UI.

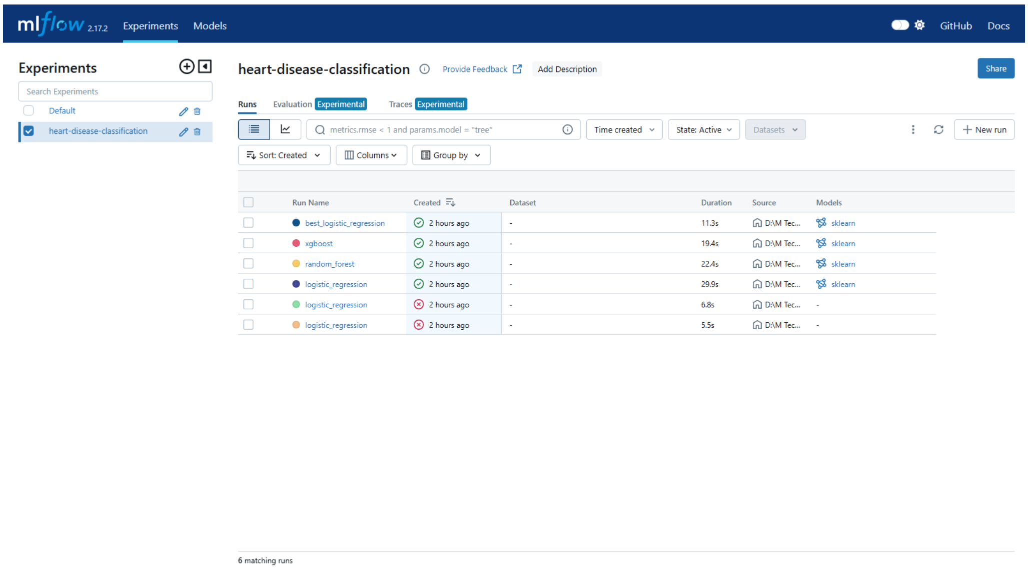

Figure 9: MLflow experiment `heart-disease-classification` — all runs (LR / RF / XGBoost) compared side by side.

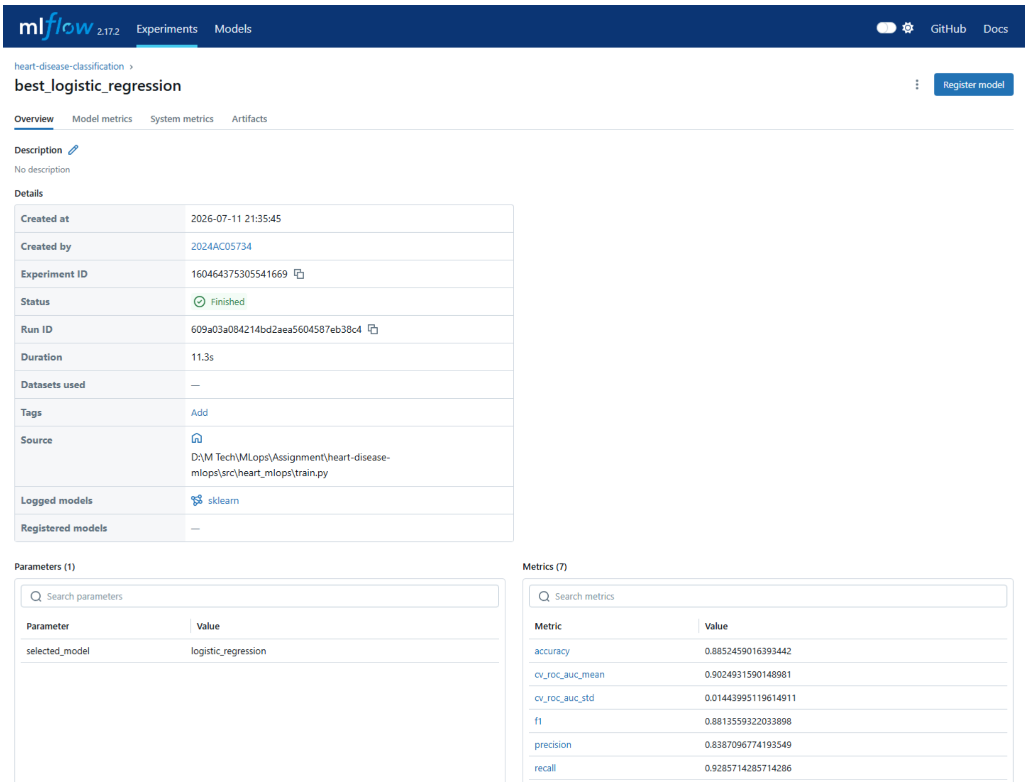

Figure 10: Best run `best_logistic_regression` — logged parameters and 7 metrics (accuracy 0.885, CV ROC-AUC 0.902, F1 0.881, precision 0.839, recall 0.929).

5. Model Packaging & Reproducibility

The final pipeline (preprocessing + classifier) is serialized to `models/model.joblib` and also registered as an MLflow model. A pinned `requirements.txt` and `pyproject.toml` allow a clean-environment install. A fixed random seed (42) makes splits and models deterministic. Because the preprocessing is inside the saved pipeline, inference is fully reproducible.

6. CI/CD Pipeline & Automated Testing

A GitHub Actions workflow (`.github/workflows/ci-cd.yml`) runs four sequential jobs on every push and pull request:

1. **lint** — flake8, black and isort checks.
2. **test** — 14 pytest unit + integration tests with coverage.
3. **train** — trains the model and uploads model, metrics and MLflow runs as artifacts.
4. **docker** — builds the image and smoke-tests the running container's `/health` endpoint.

The pipeline **fails fast**: any lint or test error stops the run and is clearly reported. Tests cover data cleaning, stratified splitting, the preprocessing transformer, model training/prediction, and all API endpoints (including validation errors and the metrics endpoint).

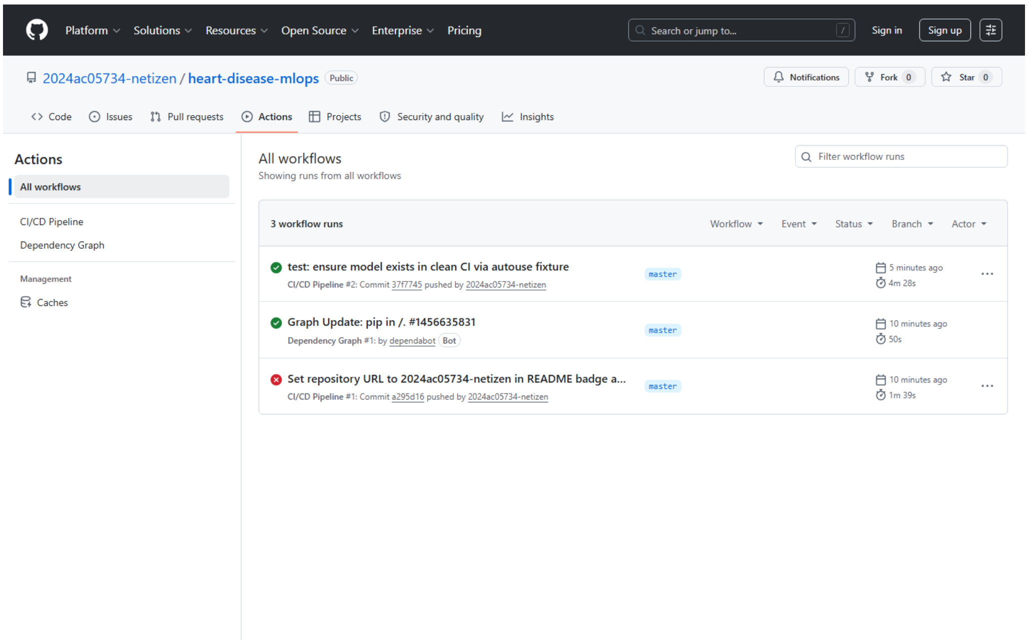

Figure 11: GitHub Actions — CI/CD Pipeline run green (lint → test → train → docker) on `2024ac05734-netizen/heart-disease-mlops`.

7. Model Containerisation

A multi-stage `Dockerfile` packages the API code, trained model, dependencies and inference pipeline. The container runs as a non-root user, exposes port 8000 and defines a health check. The FastAPI app serves `POST /predict` which accepts JSON and returns the prediction plus a confidence/probability score. `run_docker.ps1` builds, runs and smoke-tests the container with sample input in one command.

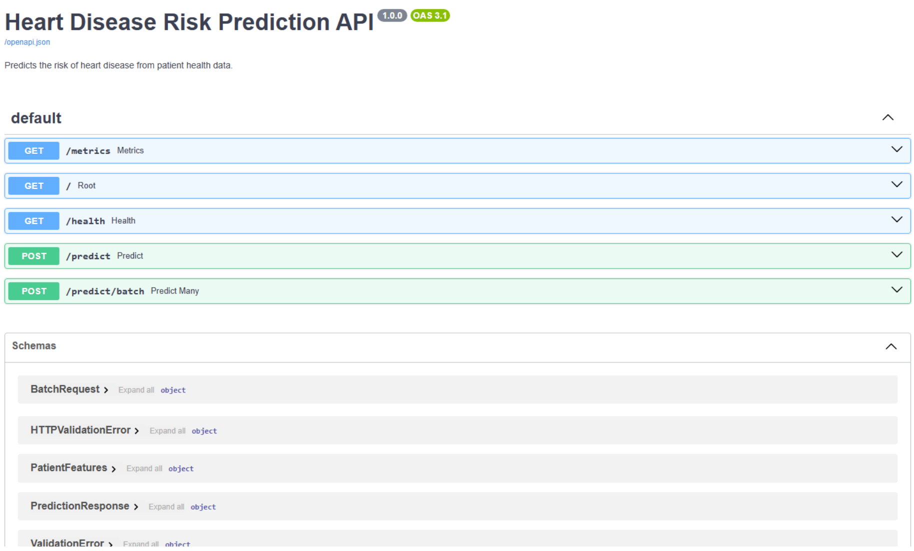

Figure 12: FastAPI Swagger `/docs` of the containerised / deployed API exposing `/health`, `/predict`, `/predict/batch` and `/metrics`.

8. Production Deployment (Kubernetes)

Kubernetes manifests in `k8s/` define a Deployment (2 replicas, resource requests/limits, liveness & readiness probes on `/health`), a LoadBalancer Service, and an Ingress. A parameterised Helm chart (`helm/heart-api`) offers the same deployment with configurable replicas, image, service type, resources and optional ingress. Deploy with `kubectl apply -f k8s/` or `helm install heart-api ./helm/heart-api` on Minikube, Docker Desktop Kubernetes, AKS, EKS or GKE.

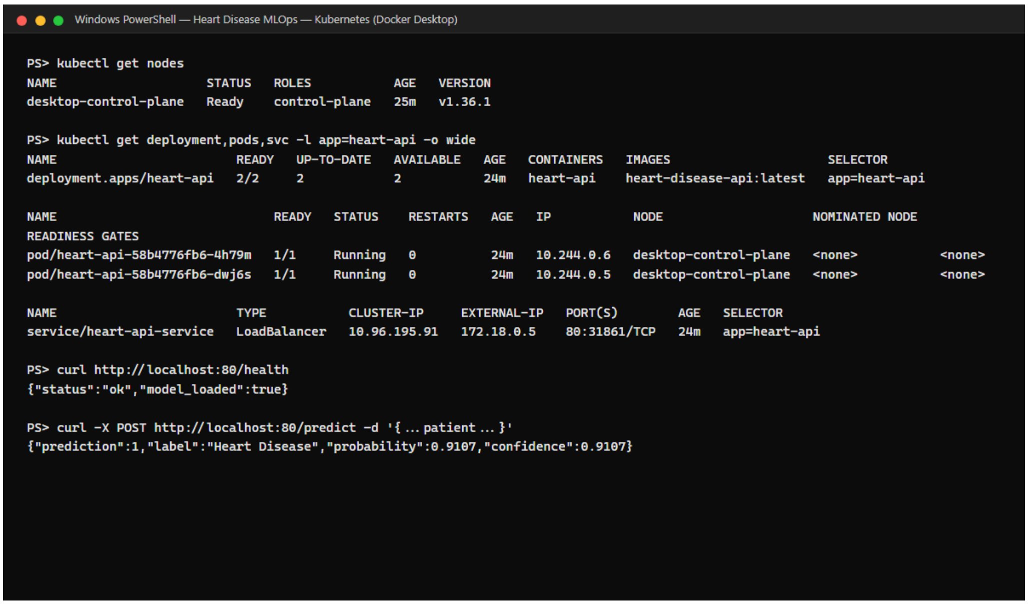

Figure 13: `kubectl get nodes/deployment/pods/svc` — 2/2 pods Running behind a LoadBalancer Service; a live `/predict` call returns "Heart Disease" (0.91).

9. Monitoring & Logging

Request/response logging (method, path, status, latency) is implemented as FastAPI middleware. The service exposes Prometheus metrics at `/metrics` — including custom counters (`heart_predictions_total` by outcome) and a latency histogram (`heart_prediction_latency_seconds`) — plus standard HTTP metrics. `docker-compose.yml` brings up Prometheus and Grafana alongside the API; a ready-made Grafana dashboard (`monitoring/grafana/heart-dashboard.json`) visualises request rate, prediction counts and p95 latency, helping detect model failures, drift and API downtime.

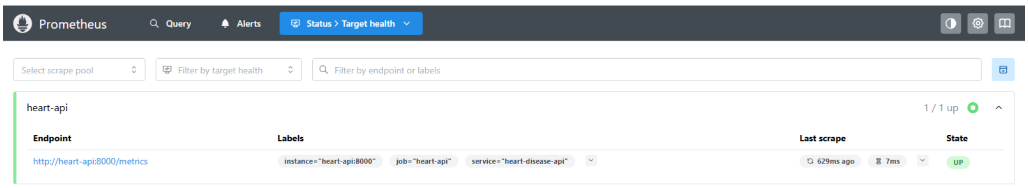

Figure 14: Prometheus scrape target `heart-api` — 1/1 UP.

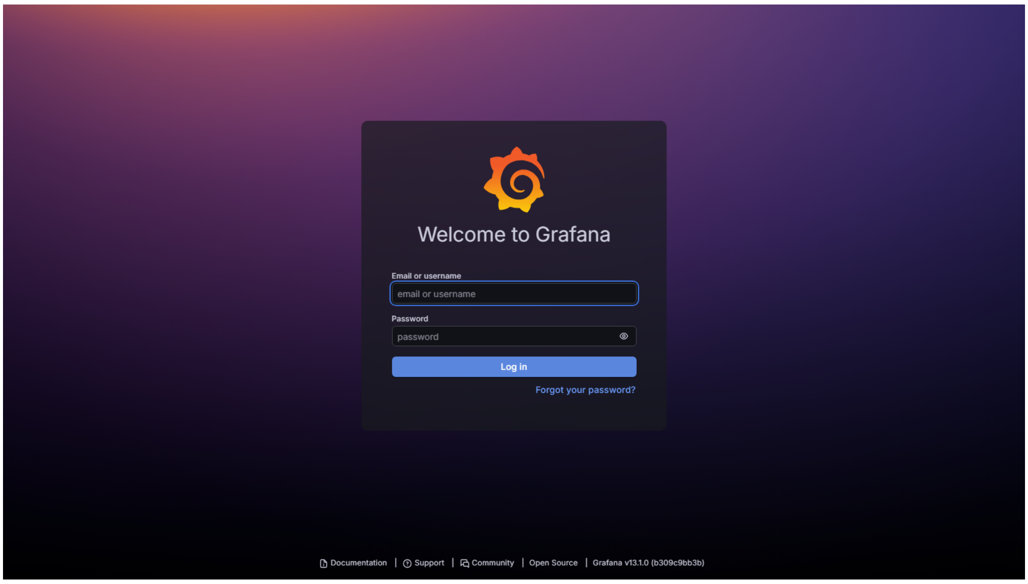

Figure 15: Grafana "Heart Disease API Monitoring" dashboard with live data — prediction counts, request rate and p95 latency panels.

10. Conclusion

The delivered system demonstrates production-ready MLOps practices: a reproducible pipeline runnable from a clean setup, tracked experiments, a tested and linted codebase, an isolated containerised service, cloud/K8s deployment artifacts and monitoring. The selected Logistic Regression model achieves a cross-validated ROC-AUC of ~0.90 and a test ROC-AUC of ~0.97, providing a strong, interpretable baseline for heart-disease risk screening.

Repository: https://github.com/2024ac05734-netizen/heart-disease-mlops

Video Walkthrough: https://github.com/2024ac05734-netizen/heart-disease-mlops/tree/master/reports — Pipeline_Walkthrough